

CV5: Shopping Checklist v Jetpack Compose - Teória

1 Cieľ cvičenia

V tomto cvičení sa budeme venovať zobrazovaniu kolekcie dát pomocou `LazyColumn`. Po absolvovaní cvičenia by ste mali rozumieť tomu, prečo sa pri väčšom počte opakujúcich položiek nepoužíva obyčajná `Column`, ale lazy zoznam, ktorý vykresľuje iba potrebné položky.

Výsledkom bude jednoduchá miniaplikácia typu nákupný zoznam. Používateľ bude vedieť pridávať nové položky, filtrovať ich podľa stavu, označovať ich ako kúpené a odstraňovať ich zo zoznamu. Aplikácia je zvolená zámerne, pretože obsahuje jednu hlavnú opakujúcu sa entitu a pritom ostáva dostatočne jednoduchá na sústredenie sa na zoznamy.

2 Čo je LazyColumn

`LazyColumn` je composable určený na zobrazovanie vertikálneho zoznamu položiek. Na rozdiel od obyčajnej `Column` nevytvára naraz všetkých svojich potomkov, ale iba tých, ktorý sú aktuálne potrební na zobrazenie. Tento princíp je dôležitý najmä pri dlhších zoznamoch.

V Jetpack Compose teda `LazyColumn` nepredstavuje iba iný spôsob scrollovania. Je to základný nástroj na prácu s kolekciami dát. Ak pracujete s opakovanými položkami, mali by ste automaticky uvažovať nad lazy zoznamom.

```
LazyColumn {
    items(shoppingItems, key = { it.id }) { item ->
        ShoppingItemCard(
            item = item,
            onToggleBought = { /* ... */ },
            onDelete = { /* ... */ }
        )
    }
}
```

3 Rozdiel medzi Column a LazyColumn

Pri `Column` môžeme často vidieť prístup založený na `forEach`, kde sa všetky položky vytvoria naraz. Takýto prístup je vhodný pri malom a statickom obsahu, ale nie je ideálny pre bežné zoznamy.

```
Column {
    shoppingItems.forEach { item ->
        ShoppingItemCard(item = item, ...)
    }
}
```

Pri `LazyColumn` zoznam dostáva špeciálny blok `items(...)` a vie s kolekciou pracovať efektívnejšie. Nie je to iba o ďalšom layoute, ale o inom spôsobe práce s opakujúcimi sa dátami.

4 Jedna hlavná entita a jej karta

Aj v tomto cvičení je dôležité postaviť aplikáciu okolo jednej hlavnej entity. V našom prípade je touto entitou `ShoppingItem`. Každá položka má identifikátor, názov a informáciu o tom, či už bola kúpená.

```
data class ShoppingItem(
    val id: Int,
    val name: String,
    val isBought: Boolean
)
```

5 Enum pre filter

Okrem samotných položiek obsahuje aplikácia aj jednoduchý filter. Ten je reprezentovaný enum triedou `ShoppingFilter`. Použitie enumu je vhodné, pretože ide o malý počet presne definovaných stavov, medzi ktorými prepíname.

```
enum class ShoppingFilter {
    ALL,
    ACTIVE,
    BOUGHT
}
```

Tento prístup je v `Compose` veľmi praktický. Stav filtra sa dá jednoducho uchovávať ako jedna hodnota a na jej základe vieme odvodiť výsledný zoznam, ktorý sa má používateľovi zobraziť.

6 Stav zoznamu a mutableStateListOf

Keďže sa zoznam v aplikácii mení, nestačí použiť obyčajný nemenný zoznam bez stavu. Vhodným riešením je `mutableStateListOf(...)`, ktoré Compose umožní sledovať zmeny v kolekcii a správne prekresľovať používateľské rozhranie.

```
val items = remember {
    mutableStateListOf(
        ShoppingItem(1, "Milk", false),
        ShoppingItem(2, "Bread", false)
    )
}
```

Týmto spôsobom môžete vidieť, že Compose nereaguje iba na zmenu jednej premennej, ale aj na zmenu kolekcie, pokiaľ je táto kolekcia súčasťou stavu.

7 Odvodený zoznam podľa filtra

Dôležitou myšlienkou tohto cvičenia je, že používateľské rozhranie sa nemusí viazať priamo na pôvodný zoznam, ale na zoznam odvodený z aktuálneho stavu. V našom prípade sa výsledný zoznam mení podľa hodnoty `selectedFilter`.

```
val filteredItems = items.filter { item ->
    when (selectedFilter) {
        ShoppingFilter.ALL -> true
        ShoppingFilter.ACTIVE -> !item.isBought
        ShoppingFilter.BOUGHT -> item.isBought
    }
}
```

Toto je veľmi dobrý príklad deklaratívneho prístupu. Namiesto ručného skrývania a zobrazovania prvkov opisujeme, aké dáta má UI dostať pri jednotlivých stavoch.

8 Pridávanie novej položky

Aplikácia obsahuje aj jednoduché pole na pridanie novej položky. Nejde o plnohodnotný formulár ako v CV3, ale o malý doplnok, ktorý umožní ukázať, že zoznam sa dá meniť aj počas behu aplikácie.

Používateľ zadá text, stlačí tlačidlo a ak pole nie je prázdne, do zoznamu sa pridá nová položka. Súčasne sa inkrementuje identifikátor a vstupné pole sa vyčistí.

```
val trimmedText = newItemText.trim()
if (trimmedText.isNotEmpty()) {
    items.add(
        ShoppingItem(
            id = nextId,
            name = trimmedText,
            isBought = false
        )
    )
    nextId++
    newItemText = ""
}
```

9 Znovupoužiteľná karta položky

Každá položka v zozname sa zobrazuje cez composable `ShoppingItemCard(...)`. Tento komponent prijíma objekt položky a callbacky pre zmenu stavu a odstránenie. Ide o priamu aplikáciu princípov z CV4. Zoznam teda nenahrádza decompozíciu UI. Naopak, dobrá práca s `LazyColumn` predpokladá, že položka zoznamu je samostatný a prehľadný composable.

```
ShoppingItemCard(
    item = item,
    onToggleBought = { /* ... */ },
    onDelete = { /* ... */ }
)
```

Tento komponent okrem názvu položky obsahuje dve akcie. Prvá prepína stav kúpené alebo nekúpené. Druhá otvorí potvrdzovací dialóg pre odstránenie.

10 Viditeľná zmena stavu položky

Aby bolo správanie zoznamu dobre pozorovateľné, zmena stavu položky sa prejaví dvoma spôsobmi. Pri označení ako kúpené sa zmení pozadie karty na zelené a text sa prečiarkne.

```
val checkedContainerColor = Color(0xFFDDF6D8)
val defaultContainerColor = MaterialTheme.colorScheme.
    surfaceVariant
```

Môžete tak vidieť, že ten istý objekt `ShoppingItem` ovplyvňuje text, farbu pozadia aj filtrovanie zoznamu.

11 Potvrdzovací dialóg pri mazaní

Na rozdiel od prepínania stavu kúpené alebo nekúpené je odstránenie položky nenávratná akcia. Preto je vhodné použiť `AlertDialog`, ktorý používateľa požiada o potvrdenie.

```
if (showDeleteDialog) {
    AlertDialog(
        onDismissRequest = { showDeleteDialog = false },
        title = { Text("Delete item") },
        text = { Text("Do you want to delete \"${item.name}\"?) },
        confirmButton = { /* ... */ },
        dismissButton = { /* ... */ }
    )
}
```

V tomto bode je užitočné ukázať, že nie každý stav musí byť globálny. Premenná `showDeleteDialog` sa týka iba jednej konkrétnej karty, preto môže zostať lokálna priamo v rámci `ShoppingItemCard(...)`.

12 key v zozname

Pri položkách v `LazyColumn` je vhodné používať parameter `key`. Ten pomáha Compose správne rozpoznať identitu jednotlivých prvkov pri zmenách v zozname.

```
items(
    items = filteredItems,
    key = { it.id }
) { item ->
    ShoppingItemCard(item = item, ...)
}
```

13 rememberLazyListState

Pre lazy zoznam je možné vytvoriť aj objekt stavu pomocou `rememberLazyListState()`. V tomto cvičení ho zatiaľ nevyužívame na pokročilé správanie, ale je vhodné ukázať, že zoznam môže mať vlastný stav podobne ako iné časti UI.

```
val listState = rememberLazyListState()
```

Neskôr sa tento objekt môže využiť napríklad pri programovom scrollovaní alebo pri reakcii na pozíciu zoznamu.

14 Empty state

Dobré používateľské rozhranie má vedieť reagovať aj na situáciu, keď po filtrovaní nezostane žiadna položka. Vtedy namiesto prázdneho zoznamu zobrazíme jednoduchý informačný blok.

```
if (filteredItems.isEmpty()) {  
    EmptyState()  
} else {  
    LazyColumn { ... }  
}
```

15 Zhrnutie

Po absolvovaní tohto cvičenia by ste mali rozumieť tomu, ako v Jetpack Compose fungujú lazy zoznamy. Mali by vedieť použiť `LazyColumn`, blok `items(...)`, parameter `key`, odvodený filtrovaný zoznam, `mutableStateListOf(...)` a samostatný composable pre jednu položku zoznamu. Zároveň by ste mali chápať, že rovnaké princípy dekompozície UI z CV4 ostávajú platné aj pri práci so zoznamami.

Toto cvičenie zároveň vytvára dobrý základ k ďalším témam. V ďalšom kroku sa rovnaký typ aplikácie môže rozšíriť o dôslednejšiu prácu s Material 3 a `Modifier`. Neskôr môže byť stav presunutý do `ViewModelu` a dáta uložené cez `DataStore` alebo `Room`.